„wirtschaft“ dornbirn · emma & eugen

Betriebshandbuch · Website und Dienst der „wirtschaft“ Dornbirn

Stand 11. November 2026

Stand 11. November 2026. Geschrieben für den Programmierer, der die Seite übernimmt. Es beschreibt, was heute läuft, wo es läuft, wie man es ausrollt und was regelmäßig zu tun ist. Die älteren Einstiegsdokumente (`README.md` , `onboarding-kollege.md` , `final-cloud-handoff.md` , alle vom August) beschreiben einen früheren Stand ohne Dienst und Wirt-Ansicht – dieses Dokument ersetzt sie als Einstieg.

Die Sprache im Code ist Deutsch, Kommentare erklären das *Warum*. Wer eine Regel ändert, ändert den Kommentar daneben mit.

---

1. Die Bausteine

baustein	was es ist	wo es läuft
Gästeseite	Statische HTML-Seiten: Start, Termine & Tickets, Mittagstisch (Reservierung), Takeaway, Locations, Agentur, Impressum, Datenschutz, Mittagskarte	Cloudflare Pages `wirtschaft-dornbirn.pages.dev` und GitHub Pages `jonasgamper-create.github.io/wirtschaft-dornbirn-test/` (derselbe Build)
Dienst	Ein Cloudflare Worker mit einem Durable Object (SQLite). Nimmt Reservierungen, Takeaway-Bestellungen, Wartelisten und Newsletter-Anmeldungen an, hält Wochenkarte und Tischplan, liest die Termine beim Ticketdienst, verschickt Mails, hat fünf Zeitpläne	`wirtschaft-reservierung.jonas-gamper.workers.dev`
Probe-Dienst	Derselbe Code als zweite Umgebung mit eigener Datenbank, ohne Mails, ohne Zeitpläne. Die Seite spricht ihn an, wenn sie mit `?probe=1` geöffnet wurde – und immer auf der Testseite `https://probe-wirtschaft-dornbirn.pages.dev` (§10)	`wirtschaft-reservierung-probe.jonas-gamper.workers.dev`
Wirt-Ansicht	Werkzeuge fürs Haus als Einzeldateien: `wirt.html` (Alltag), `kueche.html` , `uebersicht.html` , `einrichten.html` (Tischplan), `zahlen.html` , `screen.html` (Bildschirm am Eingang)	unter `/tischplan/…` neben der Gästeseite, geschützt durch den Hausschlüssel im Link (`#k=…`)

baustein	was es ist	wo es läuft
Termine	Alle Abende beider Häuser (Wirtschaft und Kulturhaus) kommen vom Ticketdienst ticketist.io. Neue Abende findet der Dienst selbst auf den eigenen Eventseiten des Hauses (§8); Kategorien und Preise liest er seit 11.11.2026 öffentlich bei Ticketist mit, die Datei im Projekt ist nur der Rückfall	`server/src/ticketist.mjs` , `site/data/termine.json` , `site/data/ticketist-preise.json`
Anfragen	Formulare auf Locations und Agentur → Dienst → Mail an `uri@eugen.family` (`ANFRAGE_MAIL` , §5) + automatische Bestätigung an den Gast; Liste in der Wirt-Ansicht	`server/src/anfrage.mjs` , `site/anfrage-senden.js`
Abgleich	Täglicher Lauf auf GitHub: steht jeder Abend der Eventseiten auf der Webseite? Mail bei Fehlschlag	`.github/workflows/termine-abgleich.yml` , `scripts/check-termine-live.mjs`
Mails	Über Brevo (SMTP-API): Bestätigungen, Absagen, Wartelisten, Tageszettel, Wochenbericht, Wochenkarte	Brevo-Konto von Jonas, Absender heute die private Adresse von Jonas (siehe §9)
Push	Web-Push an die Wirt-Ansicht („neue Bestellung“, „wieder Karten“)	VAPID-Schlüsselpaar im Dienst

Alles kostet heute nichts: Cloudflare Gratisstufe (Worker, Durable Object, Pages), GitHub (öffentliches Repository), Brevo Gratisstufe.

---

2. Konten – heute und Ziel

konto	heute	ziel nach der übergabe
GitHub-Repository `wirtschaft-dornbirn-test`	`jonasgamper-create` , öffentlich	Konto/Organisation des Kunden (Transfer ownership); ein GitHub-Konto des Hauses hat Sarah angelegt
Cloudflare (Worker, Durable Object, Pages)	Konto von Jonas, Konto-ID `76d2e486…`	eigenes Cloudflare-Konto des Kunden; Dienst dort neu ausgerollt, Bestand per Sicherung (§6) übernommen
Brevo (Mails)	Konto von Jonas – der Dienst sendet heute mit dessen Schlüssel	Brevo-Konto des Hauses: von Sarah angelegt, Einträge beider Domains gesetzt; offen *Authenticate* und ein API-Schlüssel daraus (§9). Absender `willkommen@wirtschaft-dornbirn.at`
Domain `wirtschaft-dornbirn.at`	Kunde (Zone bei Hetzner)	Kunde; empfohlen: Zone zu Cloudflare (siehe `umzug-domain-plan.md`)
Geheimnisse (Hausschlüssel, Brevo-Schlüssel, VAPID)	von Jonas erzeugt	vom neuen Betreuer **neu erzeugt**, alte gelöscht

3. Das Repository

```
site/            Quellen der Gästeseite (HTML, CSS, JS, data/, assets/)
server/          der Dienst (wrangler.jsonc, src/index.js, src/*.mjs)
scripts/         Bau- und Prüfskripte (npm run ci ruft sie alle)
docs/            Dokumentation
dist/            der veröffentlichte Gästebuild — erzeugt, nie von Hand ändern
output/tischplan/ die gebauten Einzeldateien der Wirt-Ansicht — erzeugt
```

Regeln

- Nie direkt auf `main`. Jede Änderung als Pull Request; die Action `ci.yml` läuft auf jedem PR und muss grün sein.
- Vor jedem Commit: `npm run ci`. Es baut `dist/` und `output/`, schreibt die Versionsstempel (`?v=…`) in die HTML-Dateien, gleicht CSP und Termine ab und führt rund dreißig Prüfungen aus. Was es ändert, gehört mit in den Commit.
- Beim Merge auf `main` veröffentlicht `pages.yml` den Build automatisch auf **GitHub Pages**. **Cloudflare Pages nicht** – dafür `npm run deploy:seite` (siehe §4). Das ist die eine Stelle, die man vergessen kann.
- Keine externen Skripte, Fonts, Tracker. Keine Zahlungsdaten. Die Prüfung `check:privacy` bricht ab, wenn etwas davon auftaucht.

Node 22, keine Build-Abhängigkeiten außer Dev-Werkzeugen (`qrcode`, `wrangler` per `npx`).

---

4. Ausrollen

Gästeseite

ziel	wie	wann
GitHub Pages	automatisch durch `pages.yml`	bei jedem Merge auf `main`
Cloudflare Pages	`npm run deploy:seite` (baut und lädt `dist/` hoch)	von Hand nach jedem Merge – oder das Projekt im Cloudflare-Dashboard mit dem Repository verbinden, dann automatisch

Beide zeigen denselben Build. Nach dem Umzug auf die Domain bleibt nur einer (siehe `umzug-domain-plan.md`).

Dienst

```
cd server
npx --yes wrangler@4 deploy              # Echtbetrieb
npx --yes wrangler@4 deploy --env probe  # Probe
```

`wrangler` verlangt eine Anmeldung (`npx wrangler@4 login`) im Konto, in dem der Dienst liegt. Auf Jonas' Rechner braucht es `NPM_CONFIG_CACHE=~/.wirtschaft-npm-cache` vor dem Befehl (Rechteproblem in `~/.npm`); anderswo nicht.

Nach jedem Deploy des Echtbetriebs die Ausgabe lesen: es müssen fünf Zeilen `schedule:` erscheinen. Die Gratisstufe erlaubt fünf Zeitpläne je *Konto*; die Probe hat deshalb ausdrücklich `"crons": []`. Am 17.09. hat ein Deploy der Probe die Zeitpläne des Echtbetriebs stillschweigend gelöscht – drei Tage lang gingen keine Mails hinaus. Seither: zählen.

Was beim Bauen versteckt passiert

- `sync:versions` hängt an jede CSS/JS-Referenz einen Hash (`?v=8ca3ffa7`), damit Browser nichts Altes zeigen. Eine CSS-Änderung ändert also auch die HTML-Dateien – das ist richtig so.
- `sync:csp` trägt die Dienstadressen (echt und Probe) in die `connect-src`-Zeilen von acht Seiten ein. Adresse des Dienstes ändern ⇒ `site/data/haus.json` ändern, `npm run ci`, fertig.
- `build:tischplan` baut die Wirt-Ansicht als **Einzeldateien** (alles eingebettet, CSP mit Hash). Die Dienstadressen stehen darin fest; ein nachträglich eingefügtes `<style>` wird dort von der CSP blockiert.
- `sync:events-quelle` erzeugt `site/data/events.json` aus `termine.json` – die Startseite liest beim Öffnen zusätzlich live beim Dienst.

---

5. Konfiguration

`server/wrangler.jsonc` – die Variablen

variable	bedeutung
`ALLOWED_ORIGINS`	Herkünfte, die den Dienst aus dem Browser ansprechen dürfen. Nach dem Domainumzug in der Produktion nur noch `https://wirtschaft-dornbirn.at,https://www.wirtschaft-dornbirn.at`; in der Probe bleibt `https://probe-wirtschaft-dornbirn.pages.dev` erlaubt (die Testseite).
`GAESTE_SEITE`	Adresse der Gästeseite – daraus entstehen Links in Mails, Kalendereinträgen, Push.
`DIENST_BASIS`	Eigene Adresse des Dienstes, für Links in Mails, die ohne Anfrage entstehen (Wochenkarte).
`WIRT_MAIL`	Wohin Tageszettel und Wochenbericht gehen. Seit 27.10.2026 `veranstaltungen@wirtschaft-dornbirn.at` (Hausentscheid, `docs/mailadressen-entscheidung.md`).
`TAKEAWAY_MAIL`	Wohin die Meldung jeder einzelnen Takeaway-Bestellung geht: `service@wirtschaft-dornbirn.at`. Leer heißt: es gilt `WIRT_MAIL`.
`ABSENDER_SERVICE`, `ABSENDER_WARTELISTE`, `ABSENDER_KULTURHAUS`, `EVENTS_ABSENDER`	Absender je Zweck (Reservierung/Takeaway, Warteliste „wirtschaft“, Warteliste Kulturhaus, Terminliste). **Alle leer, bis die Domains im Brevo-Konto des Hauses beglaubigt sind und dessen API-Schlüssel als `BREVO_KEY` im Dienst steht** (§9) – eine unbeglaubigte Adresse lässt Brevo gar nichts versenden. Leer heißt: es gilt `BREVO_ABSENDER`.
`ANFRAGE_MAIL`	Wohin Anfragen aus Locations und Agentur gehen: `uri@eugen.family` (Hausentscheid 03.11., live seit 10.11.2026; vorher `willkommen@`). Seit 11.11. zeigen auch der Notweg der Formulare (Dienst nicht erreichbar → das Formular öffnet eine Mail) und die Vorgabe im Dienst dorthin. „Antworten“ in dieser Mail geht direkt an den Gast (`replyTo`); die Bestätigung an den Gast trägt `replyTo` `uri@eugen.family`. `willkommen@` bleibt Kontaktadresse der Seite.

variable	bedeutung
`ALT_RESERVIERUNG`, `ALT_TAKEAWAY`	Brücke ins Altsystem; leer heißt aus. Nach der Abschaltung der alten Seite beide leer.
`OFFEN`	`"ja"` öffnet die Wirt-Ansicht **ohne** Hausschlüssel für jeden. Nur für Testphasen. Steht auf `"nein"`.
`VAPID_OEFFENTLICH`, `VAPID_KONTAKT`	Öffentlicher Teil des Push-Ausweises (kein Geheimnis).

Erzeugte Dateien im Verzeichnis. `npm run ci` baut `site/index.html` (Terminliste und Zeitstrahl), `output/tischplan/*.html` und `dist/` neu. Weil die Terminliste nur kommende Abende enthält, **ändert sich** `site/index.html` **jeden Tag von selbst** – nach einem Lauf zeigt `git status` sie als geändert, auch wenn niemand etwas getippt hat. Das ist richtig und kein Fehler: einfach mitschicken oder mit `git checkout -- site/index.html` verwerfen. Die veröffentlichte Seite ist davon nie betroffen, weil `npm run deploy:seite` ohnehin zuerst neu baut.

Die Probe-Umgebung (`env.probe`) wiederholt die Variablen, weil wrangler sie nicht vererbt. Was oben geändert wird, unten mitändern – außer bei `ALLOWED_ORIGINS`: Die Probe braucht dort weiter die Testseite.

Geheimnisse (nie in Dateien)

```
cd server
npx wrangler@4 secret put HAUS_TOKEN        # Hausschlüssel (bash schluessel.sh erzeugt einen neuen)
npx wrangler@4 secret put BREVO_KEY         # API-Schlüssel aus dem Brevo-Konto (heute der von Jonas, künftig de
npx wrangler@4 secret put BREVO_ABSENDER    # beglaubigte Absenderadresse
npx wrangler@4 secret put WIRT_MAIL_KOPIE   # Kopie von Tageszettel und Wochenbericht (cc) an die technische Bet
npx wrangler@4 secret put VAPID_PRIVAT      # privater Teil des Push-Ausweises
npx wrangler@4 secret put BREVO_SMS_ABSENDER # optional, SMS
```

Fehlen `BREVO_KEY` / `BREVO_ABSENDER`, versendet der Dienst schlicht nichts; alles andere läuft weiter. Fehlt `WIRT_MAIL_KOPIE`, gibt es keine Kopie – und so ist es heute: vorbereitet (Antwort des Hauses vom 03.11.), aber noch nicht gesetzt. Jonas setzt es selbst; ein Geheimnis, weil die Adresse privat ist und das Repository öffentlich. Die Probe hat nur `HAUS_TOKEN`.

Wird `HAUS_TOKEN` erneuert, brauchen alle Geräte im Haus einen neuen Link (`…/tischplan/wirt.html#k=<neuer Schlüssel>`). Der Schlüssel im Link wird beim ersten Öffnen im Browser gespeichert und aus der Adresszeile entfernt.

`site/data/haus.json`

Die einzige Stelle, an der die Gästeseite die Dienstadressen kennt: `api` (echt) und `probe`. Leer heißt: Dienst aus, Seite verhält sich wie eine reine Infoseite.

---

6. Daten: Durable Object, Sicherung, Fristen

Der Dienst hält alles in **einer** SQLite-Datenbank (Durable Object `Haus`), sieben Tabellen:

tabelle	inhalt
`reservierungen`	jede Reservierung als JSON (Status, Kontakt, Tisch, Verlauf)
`takeaway`	jede Bestellung
`newsletter`	Abonnenten beider Listen (Wochenkarte, Termine) mit Bestätigungs-Token
`einstellungen`	~35 Schlüssel: Tischplan, Wochenkarte (+ Entwurf), Termine-Cache, Wartelisten, Zeitsperren, Öffnungszeiten, Automatik, eigene Kennungen, Papierkorb, Zähler …
`zahlen`	Tageszähler für die Monatszahlen
`sperrliste`	Fingerabdrücke abgewiesener Absender
`mittagskarte`	die hochgeladene PDF-Karte in Teilen (BLOB)

Sicherung – der ganze Bestand als Datei (seit 30.09.):

```
# hinaus (Backup, Umzug):
curl -H "x-haus-token: $T" https://<dienst>/api/sicherung -o sicherung.json
# hinein (nur in einen leeren oder zu ersetzenden Dienst!):
curl -X POST -H "x-haus-token: $T" -H "content-type: application/json" \
     --data @sicherung.json "https://<dienst>/api/sicherung?ersetzen=1"
```

Der Import leert jede Tabelle und befüllt sie neu, in einer Transaktion; ohne `ersetzen=1` passiert nichts. Geheimnisse sind nicht in der Datei. Die Datei enthält **Gästedaten** – nicht ins Repository, nicht per Mail, nach Gebrauch löschen. Empfehlung: einmal wöchentlich sichern (ein Cron-Job auf irgendeinem Rechner mit dem Hausschlüssel genügt).

Fristen (so steht es auf der Seite, so tut es der Dienst): Reservierungen und Bestellungen werden spätestens 30 Tage nach dem Termin gelöscht; Wartelisten-Einträge nach dem Tag; Newsletter-Abonnenten bleiben bis zur Abmeldung.

Testdaten: Namen mit `TEST` sind Testdaten. Vor dem echten Start in der Wirt-Ansicht entfernen (Reservierung: „entfernen“; Takeaway: „Tag leeren“) oder per Sicherung/Import bereinigen.

---

7. Die fünf Zeitpläne

Alle in UTC; **die Hausuhr (Europe/Vienna) entscheidet im Code**, welcher Lauf was tut. Deshalb stehen für Sommer- und Winterzeit je zwei Stunden.

eintrag	hauszeit	was
`15 5 * * 1`, `15 6 * * 1`	Montag 07:15	Wochenkarte an die Abonnenten – nur, wenn seit dem letzten Versand eine neue Karte kam
`*/15 8-12 * * 1-5`	Mo–Fr 10:00–14:45	Tischerinnerungen (Push/Mail) vor der Reservierung

eintrag	hauszeit	was
`0 4,5,6,7,10,11 * * *`	06:00 · 08:00 · 12:00	06:00 und 12:00 Termine/Wartelisten beim Ticketdienst auffrischen; 08:00 Tageszettel an `WIRT_MAIL` (`veranstaltungen@`)
`0 13,14,18,19 * * 5`	Freitag 15:00 · 20:00	15:00 Wochenbericht; 20:00 Entwurf der nächsten Wochenkarte anlegen

Mehr als fünf geht auf der Gratisstufe nicht. Braucht es einen weiteren Zeitpunkt: **eine weitere Stunde in einen bestehenden Eintrag** schreiben und im Code nach Hauszeit verzweigen (so wurde 06:00/12:00 gelöst).

---

8. Termine und Ticketdienst

- **Neue Abende kommen von selbst** (seit 30.09./01.10.). Ticketist hat keine öffentliche Liste je Veranstalter. Die eigenen Eventseiten des Hauses (`QUELLSEITEN` in `ticketist.mjs` : `wirtschaft-dornbirn.at/event/` samt Blätterseiten `/page/N/` , `eugen.family/event/`) verlinken aber jeden Abend auf `ticketist.io/events/<kennung>` . `entdecke()` liest diese Listen – Blätterseiten **jedes Mal**, einzelne Abendseiten nur einmal (Merker `quellSeiten`). Der Dienst ruft das um 06:00, 12:00 und beim Knopf „Jetzt nachsehen“ auf und merkt neue Kennungen in `eigeneKennungen` vor. **Achtung Domainumzug:** Wird `wirtschaft-dornbirn.at` unsere Seite, gibt es `/event/` nicht mehr – dann `QUELLSEITEN` anpassen (eugen.family bleibt, oder eine Ticketist-Verwaltungsschnittstelle, falls verfügbar).
- **Abgleich:** Nach jeder Suche prüft der Dienst, ob jede gefundene Kennung gelesen ist; fehlt eine zwei Durchgänge lang, kommt **eine** Push-Meldung. Ergebnis in der Wirt-Ansicht über den Abenden. Von außen prüft `npm run check:live` (Eventseiten gegen Dienst und Live-Seite) – täglich 07:30 Wien als GitHub-Workflow `termine-abgleich.yml` ; schlägt er fehl, mailt GitHub dem Besitzer des Repositorys.
- **Pressefotos** neuer Abende: `npm run sync:termine` nutzt dieselbe Suche, lädt die Bilder (1200 px und `@2x` bis 3000×1500) und schreibt `termine.json` . Bis dahin zeigt die Kachel ein Ersatzbild.
- **Vorverkauf:** `salesStartAt` in der Zukunft = Zustand „Vorverkauf“ (Kachel „vorverkauf ab …“), nicht „ausverkauft“.
- Feste Kennungen stehen weiter in `KENNUNGEN` (50, Stand 01.10.); dazu die, die der Wirt in der Wirt-Ansicht von Hand einträgt.
- Der Dienst liest jede Eventseite `ticketist.io/events/<kennung>` alle 12 h (`window.event` im Quelltext: Name, Datum, Ort, Bild, Beschreibung, Verkaufsschalter), `ticketist.io/api/events/<nummer>` für die verkauften Karten und seit 11.11.2026 `ticketist.io/api/ticket-categories?eventId=<nummer>` für Kategorien, Preise und freie Karten (`holeKategorien`). Ausverkauft = Schalter zu **oder** alle Kategorien 0 frei.
- **Preise** gibt Ticketist öffentlich heraus, samt Kategorien und freien Karten (nachgeprüft am 10.11.2026). Der Dienst liest sie mit den Eventdaten – je Abend spätestens alle 12 Stunden, wie die übrigen Angaben – und liefert sie über `/api/termine` aus; Eventseite und Startseite zeigen sie. `site/data/ticketist-preise.json` ist nur noch der **Rückfall**, wenn Ticketist nicht antwortet: erzeugt von `npm run sync:termine` aus derselben Schnittstelle, nicht von Hand (das Skript liest auch alle Abende neu, lädt Bilder nach `site/assets/events/ticketist/` und schreibt `termine.json`). Ändert das Haus Preise in Ticketist, übernimmt die Seite sie von selbst. Ein Zugang bei Ticketist ist dafür nicht nötig.

- Zwei Wege zu einem Abend („dinner & comedy“ + „comedy only“) werden am Namen erkannt (`gruppiere` in `ticketist.mjs`) und auf einer Kachel gezeigt.
- Vergangene Abende bleiben sieben Tage sichtbar („schon gelaufen“), dann weg.
- Die Event-Warteliste hängt am Ticketweg. Ob ein Weg ausverkauft ist, entscheiden – wo gelesen – die Kategorien zusammen mit dem Schalter (dieselbe Regel wie die Eventseite). Der sinkende Verkaufsstand bleibt ein Hinweis auf Stornierungen, gilt aber nur, wenn auch etwas frei ist; ohne gelesene Kategorien gelten die alten Regeln. Mails an Wartende verschickt der Wirt selbst. Doku: `event-warteliste.md` .

---

9. Mails und Push

- Versand über die Brevo-SMTP-API (`server/src/mail.mjs`). Prüfweg: `GET /api/mail/pruefung?email=<Adresse>` (Hausschlüssel) fragt den Zustellstatus bei Brevo ab; `GET /api/mail/domain` zeigt die Beglaubigung. Beide fragen das Konto, dessen Schlüssel im Dienst steht – bis der Schlüssel des Hauses hinterlegt ist, also das von Jonas. Maßgeblich für *Authenticate* ist die Anzeige im Brevo-Konto des Hauses.
- **Heutiger Absender, die private Adresse von Jonas, wird von Gmail/GMX/Outlook meist abgewiesen** (ihr Anbieter verlangt per DMARC `p=reject` , fremd versandte Mails abzuweisen). Stand der Lösung: Sarah hat das Brevo-Konto des Hauses angelegt und mit Markus die Einträge für `wirtschaft-dornbirn.at` und `eugen.family` bei Hetzner gesetzt (DKIM, `brevo-code` , SPF, DMARC; von außen geprüft am 10.11.2026, `mail-absender.md`). Es fehlen, in dieser Reihenfolge: (1) *Authenticate* für beide Domains im Brevo-Konto des Hauses; (2) ein API-Schlüssel aus diesem Konto als `BREVO_KEY` , gemeinsam hinterlegt, nicht per Mail oder Chat – der Dienst sendet heute über das Konto von Jonas, und dort sind die Domains des Hauses nicht beglaubigt; (3) erst dann `secret put BREVO_ABSENDER willkommen@wirtschaft-dornbirn.at` und die Absender je Zweck (§5). Umgekehrt versendet Brevo gar nichts mehr.
- Newsletter: zwei getrennte Listen (Wochenkarte, Termine), Double-Opt-in über `/newsletter/ja` , Abmeldung `/newsletter/weg` . Abonnenten liegen im Dienst (Tabelle `newsletter`), nicht bei Brevo.
- Push: VAPID-Paar; der öffentliche Teil steht in `wrangler.jsonc` , der private als Geheimnis. Neues Paar erzeugen ⇒ alle Geräte melden sich neu an.
- **Automatische Antworten an Gäste** (alle über Brevo, alle mit dem Absender aus `BREVO_ABSENDER`): Reservierung (mit Absage-Link und .ics), Takeaway (Bestätigung, später „fertig“), Anfrage Locations/Agentur (Bestätigung mit den Angaben, seit 01.10.), Event-Warteliste (Aufnahme, „wieder Karten“), Mittags-Warteliste („Tisch frei“), Newsletter (Double-Opt-in). Ohne Brevo-Schlüssel wird nichts verschickt, alles andere läuft – Anfragen und Reservierungen stehen trotzdem in der Wirt-Ansicht.
- **Brevo-Gratisstufe: 300 Mails am Tag.** Reicht für den Alltag; ein Newsletter an viele Abonnenten kann an einem Tag darüber gehen.
- Mails direkt an `willkommen@…` laufen nicht über den Dienst. Eine automatische Antwort für `willkommen@` hat das Haus abgelehnt (Hausentscheid B9, `mailadressen-entscheidung.md`).

---

10. Wirt-Ansicht und Probemodus

- Alltag: `/tischplan/wirt.html` (Reiter: heute · karte · warteliste · haus). Zugang über den Link mit `#k=<Hausschlüssel>` ; das Gerät merkt sich den Schlüssel. Weitere Ansichten: `kueche.html` , `uebersicht.html` , `einrichten.html` (Tischplan in Zahlen), `zahlen.html` , `screen.html` .

- **Zugangsschutz heute = ein Link.** Wer ihn hat, ist drin. Nach dem Domainumzug: Cloudflare Access davor (`umzug-domain-plan.md` , §4), dann Hausschlüssel rotieren.
- **Testseite:** `https://probe-wirtschaft-dornbirn.pages.dev` ist immer Testbetrieb, auch in einem neuen Tab: eigener Testdienst, eigene Daten, keine Mails, kein Push; unten das weinrote Band „Testseite“. Das ist der Link zum Herzeigen (`testumgebung.md`), veröffentlicht mit `npm run deploy:probeseite` . Die Kopie auf GitHub Pages spricht dagegen mit dem **echten** Dienst.
- **Probemodus:** jede Seite mit `?probe=1` öffnen ⇒ dieser Tab spricht den Probe-Dienst an (eigene Datenbank, keine Mails). Gilt für den Tab, ein neuer Tab ist Echtbetrieb. Das Hinweisband ist dort seit 22.09. abgeschaltet (`BAND_ZEIGEN = istTestseite` in `site/probe.js` : nur die Testseite zeigt es) – **man sieht einem Tab nicht an, ob er im Probemodus ist.** `window.WIRTSCHAFT_PROBE` in der Konsole sagt es.
- Tischautomatik ist in beiden Diensten **aus**: jede Onlinereservierung wird angenommen, „voll“ gibt es nur durch „Tag voll melden“ oder Zeitsperren in der Wirt-Ansicht. Bewusste Entscheidung, vor dem Start nochmals prüfen (`umzug-domain-plan.md` , §6).

---

11. Was regelmäßig zu tun ist – im Haus

wann	was	wo	wenn es unterbleibt
jede Woche (bis Sonntag)	Wochenkarte der kommenden Woche eintragen oder den Freitag-Entwurf bestätigen	Wirt-Ansicht → karte	Der Dienst schreibt die alte Karte mit neuem Datum fort – Gäste sehen keine alte Woche, aber auch keine neue Gerichte
bei jedem neuen Abend	nichts – kommt von selbst; Abgleich-Zeile im Reiter warteliste prüfen	Reiter warteliste	–
nach neuen Abenden	`npm run sync:termine` , PR – bringt die Pressefotos	Repository	Kachel mit Ersatzbild
bei Anfragen	beantworten (auf die Mail an `uri@eugen.family` antworten), „erledigt“	Wirt-Ansicht → haus → Anfragen	–
bei Preisänderungen	nichts – Preise in Ticketist ändern, die Seite übernimmt sie (spätestens nach 12 Stunden); die Rückfalldatei frischt `npm run sync:termine` mit auf	Ticketist	–
täglich mittags	Reservierungen und Bestellungen abarbeiten, Wartende verständigen	Wirt-Ansicht → heute	–
bei Bedarf	Tag voll melden, Zeiten sperren, Tag absagen	Wirt-Ansicht → heute / haus	Der Dienst nimmt weiter an
wöchentlich	Sicherung ziehen (§6)	`GET /api/sicherung`	Kein Backup

---

12. Störungen

symptom	prüfen
Seite lädt, aber keine Termine / Reservierung geht nicht	`curl https://<dienst>/api/gesundheit` (erwartet `{"ok":true,…}`); Browser-Konsole: CORS-Fehler ⇒ Herkunft fehlt in `ALLOWED_ORIGINS`
Keine Mails	`GET /api/mail/pruefung?email=…`; Brevo-Konto: Absender beglaubigt? Geheimnisse gesetzt? Zeitpläne vorhanden (Deploy-Ausgabe)?
Wirt-Ansicht „Kein Dienst eingetragen“ / leer	Hausschlüssel im Gerät? (`localStorage['wirtschaft-haus-token']`); Einzeldatei neu gebaut nach Adressänderung?
Tageszettel/Wochenbericht bleiben aus	Zeitpläne zählen: `npx wrangler@4 deploy` neu ausführen und fünf `schedule:`-Zeilen sehen
Live-Logs	`cd server && npx wrangler@4 tail` (Echtbetrieb), `--env probe`
Alter Stand im Browser	`?v=`-Hash fehlt/alt ⇒ `npm run ci` nicht gelaufen; Cloudflare Pages nicht neu ausgerollt ⇒ `npm run deploy:seite`

13. Wo was steht

dokument	inhalt
`uebergabe-wirtschaft.md`	**Die Übergabe für Hannah und Wolfgang: Konten, sieben Etappen mit Rückweg, Domainumzug, Änderungen ohne KI**
`handbuch-haus.md`	**Für Wolfgang und das Team: was das Haus selbst ändert, was automatisch passiert**
`mail-domain-hosting-plan.md`	**Welche Mailadresse wo hinterlegt ist, was bei einer Anfrage automatisch passiert, Domain, Hosting-Top-3, Plan für die nächsten Jahre**
`mailadressen-uebersicht.md`	**Welche Adresse wo hinterlegt ist, alle Mail-Szenarien (Absender, Antwort-an, Empfänger), Änderungsliste mit Ort/heute/neu, Datenhaltung**
`projektstand.md`	Gesamtübersicht, offene Entscheidungen
`uebergabe.md`	Übergabeprotokoll mit Abnahme und Belegen
`umzug-domain-plan.md`	Der Umzug auf `wirtschaft-dornbirn.at`, DNS, Access
`mail-absender.md`	Brevo-Beglaubigung, die vier DNS-Einträge
`adresse.md`	Cloudflare Pages, Veröffentlichung
`testumgebung.md`	Testseite, Probe-Dienst und Probemodus
`event-warteliste.md`	Wartelisten für Abende und Mittag

dokument	inhalt
`abschaltung-alte-seiten.md`	Was an den alten Seiten hing, Weiterleitungen
`floorplan-model.md`	Tischplan-Datenmodell
`event-data.md`	Termine-Daten und Status
`launch-checklist.md`	Abhakliste für den Starttag
`docs/pdf/`	die Dokumente als PDF – neu bauen mit `node scripts/build-doku-pdf.mjs docs/<datei>.md docs/pdf/<Name>.pdf` (braucht nur Chrome)
veraltet, nur Historie: `README.md`, `onboarding-kollege.md`, `final-cloud-handoff.md`, `host-cockpit-architecture.md`, `claude-*.md`	Stand August, vor Dienst und Wirt-Ansicht